

```

import tensorflow as tf

from tensorflow import keras

from tensorflow.keras import layers

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.applications import MobileNetV2

from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

import matplotlib.pyplot as plt

import os


print("=" * 60)

print("TRAINING WITH CLEAN DATASET")

print("=" * 60)


# Use clean dataset if it exists, otherwise use original
BASE_DIR = 'dataset_clean' if os.path.exists('dataset_clean') else 'dataset'


if not os.path.exists(BASE_DIR):

    print("✗ No dataset found!")

    exit()


IMG_SIZE = 224

BATCH_SIZE = 32

EPOCHS = 30


# Data augmentation
train_datagen = ImageDataGenerator(

    rescale=1./255,

    rotation_range=20,

```

```
width_shift_range=0.2,
height_shift_range=0.2,
zoom_range=0.2,
horizontal_flip=True,
fill_mode='nearest'
)

val_datagen = ImageDataGenerator(rescale=1./255)

# Load data
print("\n 📁 Loading training data...")
train_data = train_datagen.flow_from_directory(
    os.path.join(BASE_DIR, 'train'),
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=True
)

print("\n 📁 Loading validation data...")
val_data = val_datagen.flow_from_directory(
    os.path.join(BASE_DIR, 'validation'),
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode='categorical',
    shuffle=False
)
```

```
if train_data.samples == 0:
```

```
    print("❌ No images found! Please add images to dataset_clean folder")
```

```
    exit()
```

```
class_names = list(train_data.class_indices.keys())
```

```
print(f"\n✅ Classes: {class_names}")
```

```
print(f"📁 Training: {train_data.samples} images")
```

```
print(f"📁 Validation: {val_data.samples} images")
```

```
# Build model
```

```
print("\n🏗️ Building model...")
```

```
base_model = MobileNetV2(input_shape=(IMG_SIZE, IMG_SIZE, 3), include_top=False,  
weights='imagenet')
```

```
base_model.trainable = False
```

```
model = keras.Sequential([
```

```
    base_model,
```

```
    layers.GlobalAveragePooling2D(),
```

```
    layers.Dense(256, activation='relu'),
```

```
    layers.Dropout(0.5),
```

```
    layers.Dense(128, activation='relu'),
```

```
    layers.Dropout(0.3),
```

```
    layers.Dense(len(class_names), activation='softmax')
```

```
])
```

```
model.compile(optimizer='adam', loss='categorical_crossentropy',  
metrics=['accuracy'])
```

```
# Train
```

```
print("\n 🚀 Training...")
```

```
history = model.fit(  
    train_data,  
    validation_data=val_data,  
    epochs=EPOCHS,  
    callbacks=[EarlyStopping(patience=5, restore_best_weights=True)]  
)
```

```
# Save model
```

```
model.save('model.h5')
```

```
print("\n ✅ Model saved!")
```

```
# Show results
```

```
final_acc = max(history.history['val_accuracy']) * 100
```

```
print(f"\n 🎉 Final Accuracy: {final_acc:.2f}%")
```

```
if final_acc < 70:
```

```
    print("\n ⚠️ Accuracy is low. Possible issues:")
```

```
    print(" 1. Not enough images (need 50+ per class)")
```

```
    print(" 2. Images are mislabeled")
```

```
    print(" 3. Image quality is poor")import tensorflow as tf
```

```
from tensorflow import keras
```

```
from tensorflow.keras import layers
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
```

```
from tensorflow.keras.applications import MobileNetV2
```

```
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
```

```
import matplotlib.pyplot as plt
```

```
import os

print("=" * 60)
print("TRAINING WITH CLEAN DATASET")
print("=" * 60)

# Use clean dataset if it exists, otherwise use original
BASE_DIR = 'dataset_clean' if os.path.exists('dataset_clean') else 'dataset'

if not os.path.exists(BASE_DIR):
    print("❌ No dataset found!")
    exit()

IMG_SIZE = 224
BATCH_SIZE = 32
EPOCHS = 30

# Data augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
```

```
val_datagen = ImageDataGenerator(rescale=1./255)
```

```
# Load data
```

```
print("\n 📁 Loading training data...")
```

```
train_data = train_datagen.flow_from_directory(
```

```
    os.path.join(BASE_DIR, 'train'),
```

```
    target_size=(IMG_SIZE, IMG_SIZE),
```

```
    batch_size=BATCH_SIZE,
```

```
    class_mode='categorical',
```

```
    shuffle=True
```

```
)
```

```
print("\n 📁 Loading validation data...")
```

```
val_data = val_datagen.flow_from_directory(
```

```
    os.path.join(BASE_DIR, 'validation'),
```

```
    target_size=(IMG_SIZE, IMG_SIZE),
```

```
    batch_size=BATCH_SIZE,
```

```
    class_mode='categorical',
```

```
    shuffle=False
```

```
)
```

```
if train_data.samples == 0:
```

```
    print(" ❌ No images found! Please add images to dataset_clean folder")
```

```
    exit()
```

```
class_names = list(train_data.class_indices.keys())
```

```
print(f"\n ✅ Classes: {class_names}")
```

```
print(f" 🎨 Training: {train_data.samples} images")
```

```
print(f" 🎨 Validation: {val_data.samples} images")
```

```
# Build model
```

```
print("\n 🚧 Building model...")
```

```
base_model = MobileNetV2(input_shape=(IMG_SIZE, IMG_SIZE, 3), include_top=False,  
weights='imagenet')
```

```
base_model.trainable = False
```

```
model = keras.Sequential([  
    base_model,  
    layers.GlobalAveragePooling2D(),  
    layers.Dense(256, activation='relu'),  
    layers.Dropout(0.5),  
    layers.Dense(128, activation='relu'),  
    layers.Dropout(0.3),  
    layers.Dense(len(class_names), activation='softmax')  
])
```

```
model.compile(optimizer='adam', loss='categorical_crossentropy',  
metrics=['accuracy'])
```

```
# Train
```

```
print("\n 🚀 Training...")
```

```
history = model.fit(  
    train_data,  
    validation_data=val_data,  
    epochs=EPOCHS,
```

```
callbacks=[EarlyStopping(patience=5, restore_best_weights=True)]  
)
```

```
# Save model
```

```
model.save('model.h5')
```

```
print("\n✅ Model saved!")
```

```
# Show results
```

```
final_acc = max(history.history['val_accuracy']) * 100
```

```
print(f"\n🎉 Final Accuracy: {final_acc:.2f}%")
```

```
if final_acc < 70:
```

```
    print("\n⚠️ Accuracy is low. Possible issues:")
```

```
    print(" 1. Not enough images (need 50+ per class)")
```

```
    print(" 2. Images are mislabeled")
```

```
    print(" 3. Image quality is poor")
```